

1.

```
/**
 * Removes and returns the minimum value from {@code q} according to the
 * ordering provided by the {@code compare} method from {@code order}.
 *
 * @param q
 *         the queue
 * @param order
 *         ordering by which to compare entries
 * @return the minimum value from {@code q}
 * @updates q
 * @requires <pre>
 *   q != empty_string and
 *   [the relation computed by order.compare is a total preorder]
 * </pre>
 * @ensures <pre>
 *   perms(q * <removeMin>, #q) and
 *   for all x: string of character
 *     where (x is in entries (q))
 *     ([relation computed by order.compare method](removeMin, x))
 * </pre>
 */
private static String removeMin(Queue<String> q, Comparator<String> order) {

    String min = q.front();
    Int size = q.length();

    Queue<String> temp = new queue1L<>();
    For(int I = 0; I < size; i++){
        String current = q.dequeue();
        If(order.compare(current,min) < 0){
            Min = current;
        }
        Temp.enqueue(current);
    }

    Boolean remove = false;
    For(int I = 0; I < size; i++){
        String current = temp.dequeue();
        If(!removed && current.equals(min)){
            Removed = true;
        } else {
            q.enqueue(current);
        }
    }
}
```

```

    }
    Return min;
}

2.
/**
 * Sorts {@code q} according to the ordering provided by the {@code
compare}
 * method from {@code order}.
 *
 * @param q
 *         the queue
 * @param order
 *         ordering by which to sort
 * @updates q
 * @requires [the relation computed by order.compare is a total preorder]
 * @ensures q = [#q ordered by the relation computed by order.compare]
 */
public static void sort(Queue<String> q, Comparator<String> order) {

    Queue<String> temp = new Queue1L<>();

    While(q.length()>0){
        String min = removeMin(q, order);
        Temo.enqueue(min);
    }

    While(temp.length() >0){
        q.enqueue(temp.dequeue())
    }
}

```